Assignment 3: Intelligent Cancer Diagnosis

Student Name: NICOLE LEE EN XIN
Student ID: 1820232039

---

1. Objective

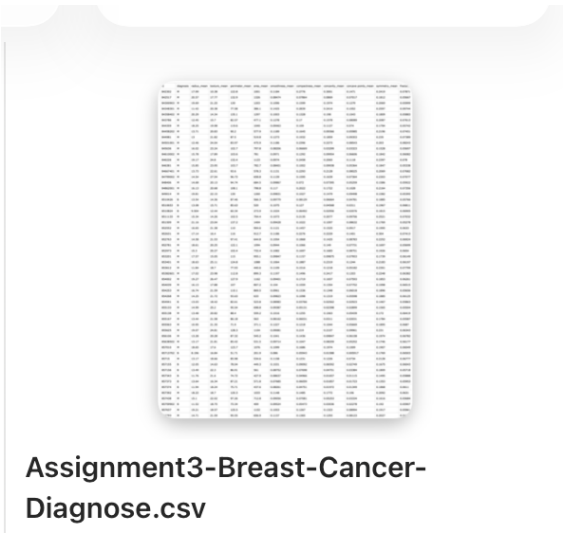
The goal of this assignment is to develop an intelligent system to classify breast tissue samples as **malignant or benign** using features extracted from fine needle aspirate (FNA) images.

Three machine learning classifiers were implemented to perform this **binary classification** task, and their performance was compared.

---

2. Dataset

- File: Assignment3-Breast-Cancer-Diagnose.csv



- Features: 30 numeric attributes computed from FNA images (e.g., radius, texture, perimeter, area, smoothness).
- Target: diagnosis — M (Malignant) or B (Benign)
- Preprocessing:
    - Converted diagnosis to numeric (1 for Malignant, 0 for Benign).
    - Dropped completely empty column (feature index 30) to avoid NaN issues.
    - Replaced missing values in numeric features using **mean imputation**.
    - Standardized features using **StandardScaler** to improve model performance.

---

3. Data Splitting

- Train/Test split: 80% train, 20% test
- Random seed: 42 for reproducibility

```
# ============================
# 5. TRAIN-TEST SPLIT
# ============================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Training samples: {X_train.shape[0]}, Test samples: {X_test.shape[0]}")
```

[13] ✓ 0.0s

```
Training samples: 455, Test samples: 114
```

---

4. Classifiers Implemented

4.1 Logistic Regression

- Pipeline: SimpleImputer ⟶ StandardScaler ⟶ LogisticRegression
- Hyperparameter tuning:
  - C = [0.01, 0.1, 1, 10]
  - Selected via **5-fold cross-validation** (GridSearchCV)
- Purpose: Linear model suitable for binary classification.

```
# SimpleImputer fills missing values
# StandardScaler scales features
log_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),  # Fill NaNs with mean
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(max_iter=1000))
])

# Hyperparameter grid
param_log = {
    'classifier__C': [0.01, 0.1, 1, 10]
}

grid_log = GridSearchCV(log_pipeline, param_log, cv=5)
grid_log.fit(X_train, y_train)  # ✅ No more NaN error

print("Best Logistic Regression Params:", grid_log.best_params_)
```

[24] ✓ 0.2s Python

4.2 K-Nearest Neighbors (KNN)

- Pipeline: SimpleImputer ⟶ StandardScaler ⟶ KNeighborsClassifier
- Hyperparameter tuning:
  - n_neighbors = [3,5,7,9]
  - Selected via **5-fold cross-validation**
- Purpose: Instance-based classifier relying on similarity in feature space.

```python
knn_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('classifier', KNeighborsClassifier())
])

param_knn = {'classifier__n_neighbors': [3, 5, 7, 9]}
grid_knn = GridSearchCV(knn_pipeline, param_knn, cv=5)
grid_knn.fit(X_train, y_train)

print("Best KNN Params:", grid_knn.best_params_)
```

[25] ✓ 0.1s Python

```
Best KNN Params: {'classifier__n_neighbors': 5}
```

4.3 Support Vector Machine (SVM)

- Pipeline: SimpleImputer → StandardScaler → SVC
- Hyperparameter tuning:
  - C = [0.1, 1, 10]
  - kernel = ['linear','rbf']
  - Selected via **5-fold cross-validation**
- Purpose: Finds the hyperplane that maximizes the margin between classes.

```python
svm_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('classifier', SVC())
])

param_svm = {'classifier__C':[0.1, 1, 10], 'classifier__kernel':['linear','rbf']}
grid_svm = GridSearchCV(svm_pipeline, param_svm, cv=5)
grid_svm.fit(X_train, y_train)

print("Best SVM Params:", grid_svm.best_params_)
```

[26] ✓ 0.1s Python

```
Best SVM Params: {'classifier__C': 0.1, 'classifier__kernel': 'linear'}
```

---

5. Model Evaluation

- Metrics used:
  - **Accuracy**
  - **Precision**
  - **Recall**
  - **F1-score**
- Evaluation was performed on the **test set**.

```python
def evaluate_model(name, model):
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(f"\n{name} Evaluation:")
    print(f"Accuracy: {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall: {rec:.4f}")
    print(f"F1-score: {f1:.4f}")

    return acc, prec, rec, f1

evaluate_model("Logistic Regression", grid_log.best_estimator_)
evaluate_model("KNN", grid_knn.best_estimator_)
evaluate_model("SVM", grid_svm.best_estimator_)
```

[27] ✓ 0.0s

```
Logistic Regression Evaluation:
Accuracy: 0.9737
Precision: 0.9762
Recall: 0.9535
F1-score: 0.9647

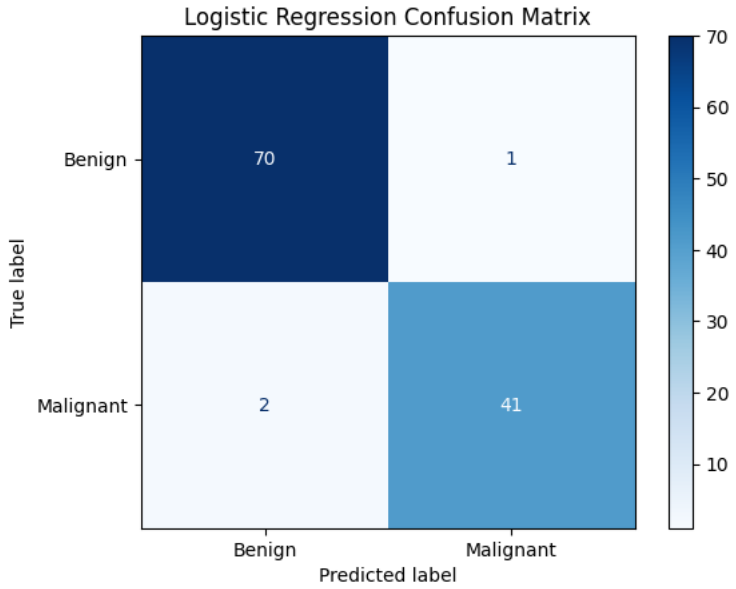
KNN Evaluation:
Accuracy: 0.9474
Precision: 0.9302
Recall: 0.9302
F1-score: 0.9302

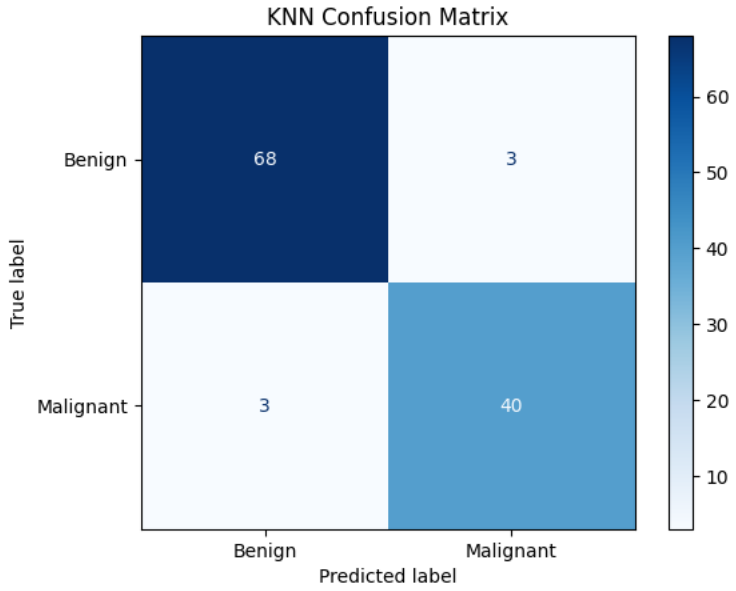
SVM Evaluation:
Accuracy: 0.9825
Precision: 1.0000
Recall: 0.9535
F1-score: 0.9762
```

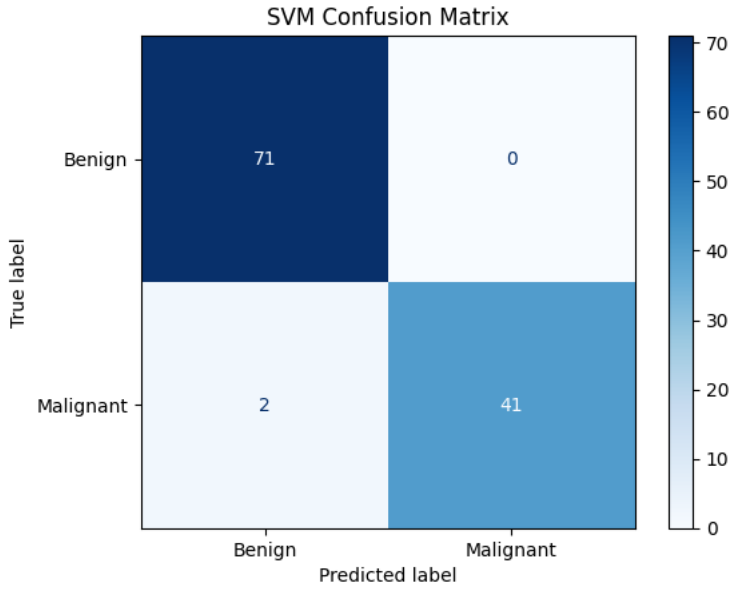
5.1 Results Table

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.9737	0.9762	0.9535	0.9647
KNN	0.9474	0.9302	0.9302	0.9302
SVM	0.9825	1.0000	0.9535	0.9762

5.2 Confusion Matrices

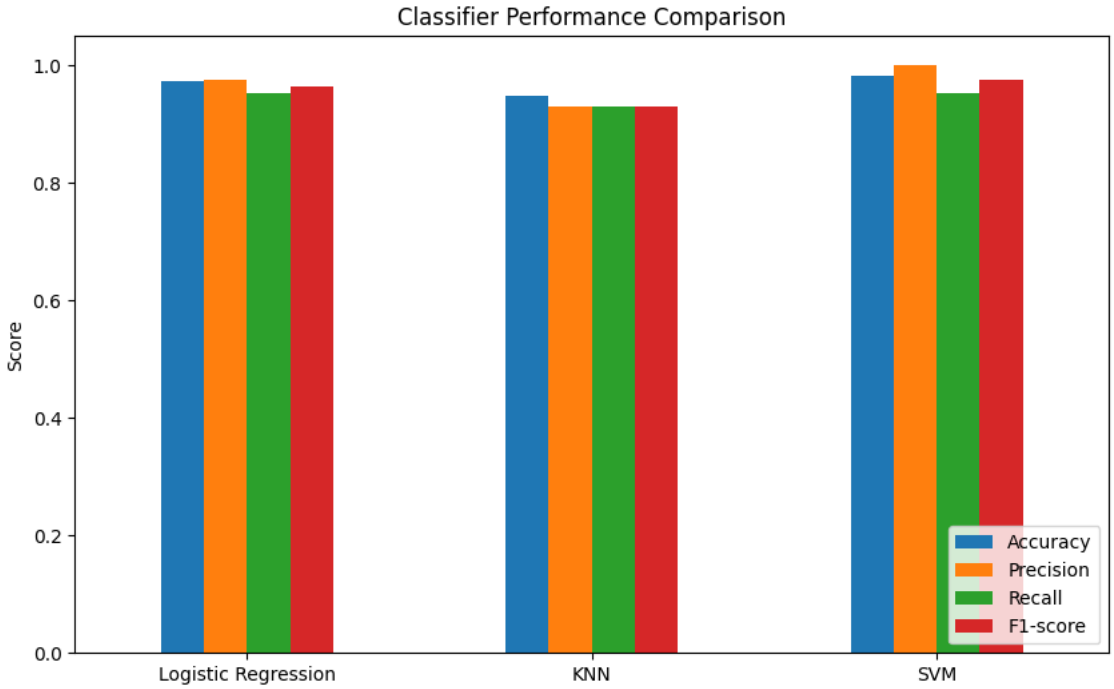

Logistic Regression Confusion Matrix
Benign
Malignant
True label
70
1
2
41
Benign
Malignant
Predicted label
70
60
50
40
30
20
10


KNN Confusion Matrix
Benign
Malignant
True label
68
3
3
40
Benign
Malignant
Predicted label
60
50
40
30
20
10


SVM Confusion Matrix
Benign
Malignant
True label
71
0
2
41
Benign
Malignant
Predicted label
70
60
50
40
30
20
10
0

- **Observation:** SVM correctly identified all malignant cases with highest precision.

5.3 Performance Comparison



- **Observation:**
  - SVM slightly outperforms Logistic Regression and KNN in accuracy and F1-score.
  - KNN performs slightly worse, but still achieves >94% accuracy.
  - All models are suitable for this binary classification task.

---

6. Conclusion

- Three classifiers were successfully implemented to diagnose breast cancer.
- Data preprocessing, including **handling missing values** and **scaling**, was critical to achieving high performance.
- **SVM** achieved the best overall performance, but Logistic Regression is simpler and also very accurate.
- KNN is effective but slightly less accurate.
- Visualizations helped in comparing the models and interpreting their results.

---

7. Future Improvements

- Use **cross-validation with separate validation set** for more robust hyperparameter tuning.

- Explore **ensemble methods** (Random Forest, Gradient Boosting) for potentially higher accuracy.
- Investigate **feature importance** to understand which FNA features contribute most to prediction.